Perfect ✅ Let's walk through a **Register + Sign-in + Access endpoint flow** using your project structure.

---

1. Register (Sign Up) Flow

📍 Example: `POST /auth/register`

Request (from frontend, e.g., React)

```
{
  "username": "manoj",
  "email": "manoj@example.com",
  "password": "12345"
}
```

Flow:

Controller (`AuthController`)

```
@PostMapping("/register")
public ResponseEntity<String> register(@RequestBody RegisterRequest
request) {
    userService.registerUser(request);
    return ResponseEntity.ok("User registered successfully!");
}
```

1.

DTO (`RegisterRequest.java`)

```
public class RegisterRequest {
    private String username;
    private String email;
    private String password;
}
```

2.

Service (`UserService`)

```
public void registerUser(RegisterRequest request) {
    User user = new User();
```

```
    user.setUsername(request.getUsername());
    user.setEmail(request.getEmail());
    user.setPassword(passwordEncoder.encode(request.getPassword()));
// hash password
    userRepository.save(user);
}
```

3.

Entity (`User.java`)

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;
    private String email;
    private String password;
}
```

4.

✅ Now the user is saved in DB with encrypted password.

---

2. Sign In (Login) Flow

📍 Example: `POST /auth/login`

Request

```
{
  "username": "manoj",
  "password": "12345"
}
```

Flow:

Controller (`AuthController`)

```
@PostMapping("/login")
```

```
public ResponseEntity<String> login(@RequestBody LoginRequest
request) {
    User user = userService.validateUser(request); // check
username+password
    String token = jwtUtil.generateToken(user.getUsername()); //
create JWT
    return ResponseEntity.ok(token);
}
```

1.

DTO (`LoginRequest.java`)

```
public class LoginRequest {
    private String username;
    private String password;
}
```

2.

Service (`UserService`)

```
public User validateUser(LoginRequest request) {
    User user = userRepository.findByUsername(request.getUsername())
                .orElseThrow(() -> new RuntimeException("User not
found"));
    if (!passwordEncoder.matches(request.getPassword(),
user.getPassword())) {
        throw new RuntimeException("Invalid password");
    }
    return user;
}
```

3.

JwtUtil (`generateToken`)

```
public String generateToken(String username) {
    return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() +
1000 * 60 * 60)) // 1 hour
            .signWith(SignatureAlgorithm.HS256, secretKey)
```

```
            .compact();
}
```

4.

✅ Response: JWT Token

```
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

---

3. Access Protected Endpoint

📍 Example: `GET /products`

Request (with token)

```
GET /products
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Flow:

1. **JwtFilter**
    - Extracts token from `Authorization` header.
    - Calls `jwtUtil.validateToken(token)` → checks signature + expiry.
    - If valid → set authentication in `SecurityContext`.

```
if (userName != null && jwtUtil.validateToken(token)) {
    UsernamePasswordAuthenticationToken authToken =
            new UsernamePasswordAuthenticationToken(userName, null,
null);
    SecurityContextHolder.getContext().setAuthentication(authToken);
}
```

2.

Controller (`ProductController`)

```
@GetMapping("/products")
public List<Product> getProducts() {
    return productService.getAllProducts();
```

```
}
```

3.

✅ If JWT is valid → user gets product list.
❌ If JWT is invalid/expired → `403 Forbidden`.

🔑 Summary of Flow

- **Register:** save user with encrypted password.
- **Login:** verify credentials → return JWT token.
- **Use JWT:** client adds token in `Authorization` header for every request.
- **JwtFilter + SecurityConfig:** validate token on each request (stateless).
- **Protected endpoints:** only accessible with valid JWT.